MMAUG / AI & DEVOPS BOOTCAMP

# PROMPT CRAFT FOUNDATIONS

Prompt Engineering, Context Engineering & AI Evaluation
A hands-on workshop for beginners

*Olawale Bello | Azure Architect*

**21-10-2026**

Scan to join the Livestream

**A Global Community for AI, Cloud, Infra, DevOps and Tech Enthusiasts**

*Learn. Innovate. Build.*

**mmaug.com**

# Meet the speaker

*A short introduction that connects your experience to the session*



**Olawale Bello**

Azure Cloud Architect

**An experienced IT professional with a career spanning over 18 years, including more than 8 years specializing in Microsoft Azure cloud technologies. Having held roles such as Deployment Manager, Azure Solution Architect, and Azure Architect, I am passionate about bridging cloud computing and Artificial Intelligence to solve real-world business challenges.**

- Experience: 18+ years in IT, including 8+ years in Microsoft Azure cloud technologies.
- Expertise: Cloud Architecture, Infrastructure Modernization, Microsoft Azure, and AI Solutions.
- Passion: Making AI accessible and helping people apply it confidently to real-world challenges.

CONNECT

**https://www.linkedin.com/in/olawale-bello-aa081335/**

# Who this session is for

*No coding background required - learn by trying simple, practical examples.*

ST

## Students

Building an AI-ready skillset alongside a degree or bootcamp.

CC

## Career changers

Moving into tech or AI-adjacent roles from another field.

EC

## Early-career technologists

Developers, analysts, or support staff adding AI to their toolkit.

BG

## Curious beginners

Anyone who wants to move from 'using ChatGPT' to using it well.

XP

## Experienced practitioners

Learn practical prompting, grounding and evaluation with everyone.

# Session roadmap

| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 5 min | 10 min | 15 min | 15 min | 15 min | 45 min | 15 min |
| Welcome | How LLMs read instructions | Prompt engineering + Lab | Context engineering + Lab | AI evaluation + Lab | Capstone lab | Wrap-up & pathways |

**PRACTICE** Everyone follows the same beginner-friendly lab activities.

# By the end of today, you will be able to

1. Write clear, structured prompts using proven techniques (role, format, examples, chain-of-thought).
2. Design context - the information surrounding a prompt (documents, memory, tools) - so an AI answers accurately.
3. Evaluate AI output systematically, using human review and automated checks, instead of guessing.
4. Navigate a real AI demo environment (Azure AI Foundry or an equivalent) to build and test a small AI workflow.

**PRACTICE** Practise with two simple examples: Nimbus Notes and Brightpath Logistics.

# Why these skills matter right now

Prompting, context design, and evaluation are becoming baseline literacy - not just for engineers.

## Entry point

AI tools are now embedded in everyday work apps - email, spreadsheets, docs, code editors.

## Force multiplier

The same model gives very different results depending on how it's asked - that gap is a learnable skill.

## Quality gate

Teams that ship AI features increasingly require review and evaluation before release, not after.

MODULE 1 · LESSON

# How large language models read instructions

The mental model you need before writing a single prompt.

# Three ideas before you prompt anything

A

## Tokens, not words

Models read text broken into token chunks - this is why very long or oddly formatted input can behave unexpectedly.

B

## A limited context window

Everything the model 'knows' for this reply - instructions, history, documents - must fit in one window, measured in tokens.

C

## Prediction, not lookup

The model predicts the most likely next tokens based on your input; it has no memory or database to consult unless you provide one.

MODULE 2 · LESSON + LAB

# Prompt engineering

Turning a vague request into instructions a model can follow reliably.

# Anatomy of a strong prompt

| | |
|---|---|
| **Role** | Who should the model act as? ("You are a patient tutor...") |
| **Task** | What exactly do you want done, stated as an instruction, not a topic? |
| **Context** | What background, data, or constraints does it need to get this right? |
| **Format** | What should the output look like? Length, structure, tone? |
| **Examples** | One or two sample inputs/outputs to anchor the style (few-shot). |

# Four techniques worth knowing cold

## Zero-shot

Ask directly, no examples.

BEST FOR

*Quick, well-known tasks.*

## Few-shot

Show 2–3 examples of the pattern you want.

BEST FOR

*Consistent formatting or tone.*

## Chain-of-thought

Ask the model to reason step by step before answering.

BEST FOR

*Multi-step logic, math, analysis.*

## System prompting

Set persistent rules and persona up front.

BEST FOR

*Apps, assistants, repeated tasks.*

# Rewrite a weak prompt three ways

## GOAL

Take one vague prompt and progressively improve it using role, format, and few-shot examples - then compare outputs side by side.

1. Start from the vague prompt provided ("write something about our product").
2. Version 1: add role + task + format. Run it and read the output.
3. Version 2: add 2 examples of the tone/style you want (few-shot). Run it.
4. Version 3: add a chain-of-thought instruction for a reasoning task. Run it.
5. In pairs, compare all three outputs and note what changed and why.

*Demo environment: Azure AI Foundry chat playground, Google AI Studio, Anthropic Workbench, or OpenAI Playground - any one works for this lab.*

MODULE 3 · LESSON + LAB

# Context engineering

Prompting decides how you ask. Context engineering decides what the model has to work with.

# Prompt engineering vs. context engineering

### PROMPT ENGINEERING

The single instruction: wording, structure, examples, tone. You control what's typed into the box.

### CONTEXT ENGINEERING

Everything else the model sees: retrieved documents, chat history, tool outputs, system rules, memory. You design the pipeline that assembles this.

*Rule of thumb: if the model "doesn't know" something it should, that's usually a context problem - not a prompting problem.*

# Four building blocks of context

1

## Retrieval (RAG)

Fetch relevant documents or data and insert the most relevant snippets into the prompt.

2

## Memory

Carry forward summaries of past turns or sessions instead of the full raw history.

3

## Tools

Let the model call a calculator, search, or API and read the result back as context.

4

## Window management

Trim, summarize, or prioritize what's included so the most relevant material fits.

# Ground a chatbot in a document

## GOAL

Connect the Nimbus Notes FAQ to a chat assistant so it answers only from that source, and see what happens when context is missing.

1. Open the provided one-page Nimbus Notes FAQ and the chat playground.
2. Ask a question the FAQ answers, without attaching it. Note any guessing.
3. Attach or paste the FAQ as context, then ask the same question again.
4. Ask a question the FAQ does NOT cover - check if it invents an answer.
5. Add one instruction line: "Only answer from the provided document; say so if it's missing." Re-test.

*Demo environment: Azure AI Foundry (File search tool), Google AI Studio ("+" icon, PDF), or paste the text into the system prompt in any tool.*

MODULE 4 · LESSON + LAB

# AI evaluation

How to know if an AI system is actually good - before your users find out for you.

# Why "it looks right" isn't enough

| | |
|---|---|
| **Hallucination** | Confident, fluent answers that are simply false or fabricated. |
| **Inconsistency** | The same question, asked twice, gets meaningfully different answers. |
| **Bias & fairness** | Outputs that perform worse, or differently, for some groups or phrasing. |
| **Drift** | Quality that quietly degrades as a model, prompt, or data source changes over time. |

# A practical evaluation toolkit

## Human review

People read and score outputs against a rubric.

BEST FOR

*High-stakes or ambiguous tasks.*

## Golden datasets

A fixed set of test inputs with known-good answers, run every time something changes.

BEST FOR

*Regression testing.*

## Automated metrics

Rule-based or statistical checks: exact match, keyword presence, length, format validity.

BEST FOR

*Fast, structured tasks.*

## LLM-as-judge

A second model scores the first model's output against your criteria.

BEST FOR

*Scaling human review cheaply.*

# How an LLM judge works

*A judge doesn't answer the question - it grades an answer someone else already gave.*

**QUESTION**

The test question you asked.

**SOURCE DOCUMENT**

The Nimbus Notes FAQ (the ground truth).

**ASSISTANT'S ANSWER**

Copied exactly as it came out.

**JUDGE PROMPT**

- Fresh session
- Four labels, each defined
- "Use only the document"
- Strict output format

**VERDICT**

- Correct
- Partially correct
- Incorrect
- Hallucinated

*+ one-sentence reason*

*Same model in a fresh chat works. A different model is better when you have one. Always spot-check the judge against your own scoring.*

# Evaluate five assistant answers with a scorecard

## GOAL

Create a small evaluation set for the assistant from Lab 2, run it, and score the results using both a human rubric and an LLM-as-judge prompt.

1. Write 5 test questions: 3 answerable from the document, 2 that are not.
2. Run all 5 through your assistant and record each output.
3. Score each with a simple rubric: correct / partially correct / incorrect / hallucinated.
4. Record scores for accuracy, grounding, instructions, format and missing information.
5. Flag any invented facts, and discuss one improvement to the prompt or context.

*Demo environment: Azure AI Foundry Evaluation tab if you have it; otherwise score by hand with the worksheet (always works, including on OpenAI Platform).*

CAPSTONE LAB

# Build and evaluate a mini assistant

Bring prompting, context, and evaluation together in one small project.

# Onboarding Helper for Brightpath Logistics

**VERSION 1**

*"You are a helpful assistant for new employees at Brightpath Logistics. Answer their questions about onboarding."*

**3 / 5**

Sick days and remote work were invented - fluent, specific, and wrong.

**VERSION 2**

*"...Answer using only the attached fact sheet. Keep answers to 2 - 3 warm, plain sentences. If it isn't covered, say so and point to HR."*

**5 / 5**

Same context, same questions. Only the system prompt changed.

*The weak point wasn't the fact sheet - it was the missing instruction for what to do when the answer isn't there.*

# Pick a mini use case and ship it

**Choose ONE:**

- **Study buddy for a course topic**
- **FAQ assistant for a small business**
- **Onboarding helper for new hires**

1. Write a system prompt: role, task, tone, and format for your assistant.
2. Provide it context: a short document, FAQ, or set of facts (10 - 20 lines) it should rely on.
3. Write 5 golden test questions and expected answers - at least one the document doesn't cover.
4. Run your tests, score the results, and fix one weak point (prompt or context).
5. Be ready to share: your prompt, your trickiest test case, and one thing you'd improve next.

# Where we're building today

*Any of these work - pick based on what accounts you already have.*

## Azure AI Foundry

Primary

Chat playground, File search for document grounding, and a built-in Evaluation tab - one place covers all three labs.

## Google AI Studio

Altemative

Free browser playground. Choose a non-preview Flash model (e.g. Gemini 2.5 Flash). Attach the FAQ with the "+" icon, as a PDF.

## Anthropic Console

Altemative

Clean Workbench with prompt versioning. Paste the FAQ into the system prompt. Needs a funded API balance.

## OpenAI Platform

Altemative

Playground for Labs 1 - 2. Its Evals dashboard is being retired (shutdown planned 30 Nov 2026), so score Lab 3 by hand.

# Key takeaways

1. A prompt is an instruction, not a topic - role, task, format, and examples do most of the work.
2. If the model "doesn't know," fix the context (documents, memory, tools) before rewriting the prompt.
3. Never trust a demo you haven't tested - build a small golden set and score it, every time.
4. These three skills compound: better context makes evaluation easier; evaluation reveals what prompts to fix.

# Where to go next

## This week

Rebuild today's exercise using a topic from your own coursework, job, or job search.

## This month

Read up on retrieval-augmented generation (RAG), try one managed vector-search tutorial, and add a regression gate to something you build.

## This quarter

Explore the companion track: Responsible AI, privacy, safety & human review.

# Questions & discussion

Prompt Craft Foundations - Prompt Engineering, Context Engineering & AI Evaluation